



spring

et

JUnit

1

JUnit et Spring

2

- ✓ Rappels JUnit
- ✓ Spring et JUnit
- ✓ Spring et Mockito
- ✓ Spring Boot 3.1, JUnit et Docker

Les Tests : objectifs

- Valider le comportement du code
- Empêcher les régressions
- Valider les impacts de modifications
- Faciliter la maintenance et l'évolution
- Il existe même des techniques de développement piloté par les tests (Test Driven Development)

Des niveaux et des phases

4

DDI

- Tests unitaires : ciblent le niveau opération. Ils prennent chaque méthode de chaque objet et vérifient que tout fonctionne correctement.
- Tests d'intégration : ciblent le niveau module. Ils vérifient les interactions entre les composants unitaires, les différents modules et dans leur environnement exploitation définitif. Ils permettent de mettre en évidence des problèmes d'interfaces.

DMOP

- Tests fonctionnels : ciblent le niveau de granularité du cas d'utilisation ou scénario. Ils permettent de vérifier la conformité de l'application développée avec le cahier des charges initial. Ils sont donc basés sur les spécifications fonctionnelles et techniques.
- Tests de non régression : permettent de vérifier que les modifications du code source n'ont pas altérées le fonctionnement de l'application.
- Tests métiers : cible le niveau processus et consiste à simuler l'exécution d'un processus avec l'application / le logiciel.

Les outils de tests de base

- Pour chaque problématique de tests on peut trouver un ou plusieurs outils de base, un ou plusieurs frameworks
- JUnit : pour les tests unitaires
 - Intégrer dans les IDE, la référence aujourd'hui
- Mockito : pour les bouchons
 - Surcharger le comportement de certains éléments sur lesquels on a pas la main
- JMeter : permet d'enregistrer des comportements et de les rejouer plusieurs fois en même temps
 - Idéal pour les tests de charge Web ou d'IHM
- Selenium : permet de tester des scénariis d'IMH
 - Idéal pour les tests fonctionnels

JUnit (1)

- Framework de référence pour réaliser des tests unitaires.
- Gratuit, et téléchargeable <http://junit.org/>
- Simple, on écrit une classe Java on utilise des annotations pour indiquer les tests et on lance le tout dans son IDE
- Le plus difficile n'est pas dans l'écriture du code mais dans sa réflexion
 - Je veux tester quoi ? pourquoi ?
 - Le comment est simple

JUnit (2)

- L'écriture des cas de tests
- Création d'une instance de la classe et de tout autre objet nécessaire aux tests
- Appel de la méthode à tester avec les paramètres du cas de tests
- Comparaison du résultat attendu avec le résultat obtenu : en cas d'échec, une exception est levée
 - Il est important de se souvenir lors de l'écriture de cas de tests que ceux-ci doivent être indépendants les uns des autres. JUnit ne garantit pas l'ordre d'exécution des cas de tests puisque ceux-ci sont obtenus par introspection.

JUnit (3)

- Deux versions de JUnit
 - 4 : Pour Java ≤ 7
 - 5 : depuis septembre 2017, pour Java ≥ 8
- Les deux versions sont rétro compatibles
- JUnit 5 est officiellement supporté par Spring à partir de **la version 2.2.x de Spring Boot = Spring 5.2.x**
 - ~ Septembre 2019

Rappel : Une classe de Test - Visibilité

- Contrairement à JUnit 4, toutes les classes et méthodes de tests doivent avoir une visibilité **package** (c.a.d pas de mot clef devant la classe et les méthodes)
- Ainsi, les classes et les méthodes de tests **doivent** avoir une visibilité **package**
 - Contrairement à JUnit 4, ou la bonne pratique est de leur donner une visibilité *public*
- Il n'est pas possible qu'elles soient *private*, vous avez le droit d'utiliser *protected*

Rappel : Une classe de Test - Méthodes

- Le nom de la méthode qui représente votre test n'a techniquement aucune importance.
 - **Cependant**, il est conseillé de lui donner un nom qui explique sur quoi porte le test et de la faire commencer par **testXxxx**
 - En JUnit 5 vous pouvez aussi utiliser l'annotation de méthode **@DisplayName** afin de fournir une information qui sera affichée dans les CR de tests

Rappel : Annotation @Test

```
package fr.exemple;
import org.junit.jupiter.api.*;
import static org.junit.jupiter.api.Assertions.*;
import fr.exemple.Personne;

class JUnit5PersonneTest {

    @Test
    void testNom1() {
        final String nom = "Dupont", prenom = "Jean";
        final int age = 25;
        Personne p = new Personne(nom, prenom, age);
        Assertions.assertEquals(nom, p.getNom(), "Les deux noms doivent être identiques");
        // Ou si vous avez fait l'import static
        // assertEquals(nom, p.getNom(), "Les deux noms doivent être identiques");
    }

    @Test
    void testNom2() {
        final String nom = "Dupont", prenom = "Jean";
        final int age = 25;
        Personne p = new Personne(null, prenom, age);
        p.setNom(nom);
        Assertions.assertEquals(nom, p.getNom(), "Les deux noms doivent être identiques");
    }
}
```

Visibilité *package*
pour les classes et
méthodes

Rappel : JUnit - Annotations @BeforeEach et @AfterEach



12

- En JUnit 5, les annotations **@Before** et **@After** se transforment en
 - **@BeforeEach** : à placer sur une méthode. Cette dernière sera appelée **avant** chaque **@Test**
 - ❑ <http://junit.org/junit5/docs/current/api/org/junit/jupiter/api/BeforeEach.html>
 - **@AfterEach** : à placer sur une méthode. Cette dernière sera appelée **après** chaque **@Test**
 - ❑ <http://junit.org/junit5/docs/current/api/org/junit/jupiter/api/AfterEach.html>

Rappel : JUnit - Annotations @BeforeAll et @AfterAll



13

- En JUnit 5, les deux annotations **@BeforeClass** et **@AfterClass** se transforment en
 - **@BeforeAll** : à placer sur une méthode. Cette dernière sera appelée avant le tout premier **@Test**
 - ❑ <http://junit.org/junit5/docs/current/api/org/junit/jupiter/api/BeforeAll.html>
 - **@AfterAll** : à placer sur une méthode. Cette dernière sera appelée après le dernier **@Test**
 - ❑ <http://junit.org/junit5/docs/current/api/org/junit/jupiter/api/AfterAll.html>
- **Les méthodes annotées doivent toujours impérativement être static.**

Rappel : JUnit - Le test de la levée d'exceptions



14

- En JUnit 5 l'annotation **@Test** n'a plus de paramètre *expected*.
- Pour remplacer cet usage vous avez la méthode **Assertions.assertThrows**

```
68+ * Test.[]
73- @Test
74 void testAuthentifierKo1() throws Exception {
75     final String login = "dfd";
76     final String pwd = "df";
77     Assertions.assertThrows(UtilisateurInconnuException.class, () -> {
78         this.authService.authentifier(login, pwd);
79     });
80 }
```

Spring et JUnit

- Spring complète JUnit avec un certain nombre d'annotations qui vont nous permettre de
 - charger le contexte
 - gérer l'aspect transactionnel (nous y reviendrons plus tard)
- Nous allons pouvoir dans nos tests faire de l'injections de dépendances
 - Via @Autowired
- Notre code de test va donc se simplifier grandement

Spring avec JUnit

- Vous pouvez aussi faire usage des annotations de méthodes
 - `@org.springframework.test.context.transaction.BeforeTransaction`
 - `@org.springframework.test.context.transaction.AfterTransaction`
- Par exemple, elles peuvent vous permettre d'initialiser la base avec des données de votre choix

Spring < 5 et JUnit 5 : chargement du contexte



17

➤ Dans le cas du chargement des fichiers XML

```
@ExtendWith(SpringExtension.class)
@ContextConfiguration(name = "applicationContext", locations = { "classpath*:spring/*-context.xml" })
class maClasseDeTest {
```

➤ Dans le cas du chargement des classes annotées

```
@ExtendWith(SpringExtension.class)
@ContextConfiguration(classes = { MaConfiguration.class }, loader = AnnotationConfigContextLoader.class)
class maClasseDeTest {
```

➤ Plus besoin du @RunWith qui est remplacé par @ExtendWith

Spring >= 5 et JUnit 5 : chargement du contexte



18

➤ Dans le cas du chargement des fichiers XML

```
@RunWith(SpringRunner.class)
@SpringJUnitConfig(locations = { "classpath*:spring/*-context.xml" })
class maClasseDeTest {
```

➤ Dans le cas du chargement des classes annotées

```
@RunWith(SpringRunner.class)
@SpringJUnitConfig(classes = { MaConfiguration.class })
class maClasseDeTest {
```

➤ Plus besoin du @RunWith

➤ @SpringJUnitConfig remplace @ContextConfiguration

Spring >= 5 et JUnit 5 : chargement du contexte

➤ Dans le cas du SpringBoot 2.2+ (en JUnit 5)

```
@RunWith(SpringRunner.class)
@SpringBootTest
class maClasseDeTest {
```

➤ Plus besoin du @RunWith

- Attention : respectez bien la contrainte sur la position de votre classe annotée en @SpringBootApplication

➤ Cette annotation, @SpringBootTest doit être positionnée au moins une fois

- Il n'est pas obligatoire de la placer sur chaque classe de test

Spring >= 5 et JUnit 5 : chargement du contexte

➤ Dans le cas du SpringBoot 2.2+ (en JUnit 5)

```
@RunWith(SpringRunner.class)
@SpringBootTest({ "MaClassSpringBoot.class" })
class maClasseDeTest {
```

- Si jamais Spring Boot a des difficultés pour trouver votre classe de configuration vous pouvez lui indiquer son nom
 - Cela ne devrait pas se produire si vous respectez bien votre structure de package

Spring chargement du contexte Web

➤ Dans le cas du chargement des fichiers XML

```
@RunWith(SpringRunner.class)
@WebAppConfiguration
@ContextConfiguration(name = "applicationContext", locations = { "classpath*:spring/*-context.xml",
"file:src/main/webapp/WEB-INF/spring-servlet.xml" })
class maClasseDeTest {
```

➤ Dans le cas du chargement des classes annotées

```
@RunWith(SpringRunner.class)
@WebAppConfiguration
@ContextConfiguration(classes = { MaConfiguration.class, MaConfigurationWeb.class }, loader =
AnnotationConfigWebContextLoader.class)
class maClasseDeTest {
```

Utilisation d'un profile

- Vous pouvez faire usage directement de l'annotation `@ActiveProfiles("nonDuProfil")`

➡ Exemple ici, utilisation du profil "test"

```
@RunWith(SpringRunner.class)
@WebAppConfiguration
@ContextConfiguration(classes = { MaConfiguration.class, MaConfigurationWeb.class }, loader =
AnnotationConfigWebContextLoader.class)
@ActiveProfiles("test")
class maClasseDeTest {
```

Spring MVC : JUnit – Avec MockMVC

- On peut utiliser les Mock Spring pour tester nos WS sans les déployer sur un serveur JEE
- Avec cette approche, il n'est plus nécessaire de déployer quoi que ce soit
 - Tests plus simples à écrire (pas de nom de serveur, ...)
 - Les tests seront lancés automatiquement avec nos tests unitaires

Spring MVC : JUnit – Avec MockMVC

- Dans un premier temps il faut légèrement modifier la déclaration de nos tests JUnit en complétant les annotations :

```
6 package com.banque.test.rest;
7
8 import org.apache.logging.log4j.LogManager;
9 import org.apache.logging.log4j.Logger;
10 import org.junit.Assert;
11 import org.junit.Before;
12 import org.junit.Test;
13 import org.junit.runner.RunWith;
14
15 import org.springframework.beans.factory.annotation.Autowired;
16 import org.springframework.http.MediaType;
17 import org.springframework.test.context.ContextConfiguration;
18 import org.springframework.test.context.junit4.SpringJUnit4ClassRunner;
19 import org.springframework.test.context.web.WebAppConfiguration;
20 import org.springframework.test.web.servlet.MockMvc;
21 import org.springframework.test.web.servlet.ResultActions;
22 import org.springframework.test.web.servlet.request.MockMvcRequestBuilders;
23 import org.springframework.test.web.servlet.result.MockMvcResultMatchers;
24 import org.springframework.test.web.servlet.setup.MockMvcBuilders;
25 import org.springframework.web.context.WebApplicationContext;
26
27 import com.banque.web.model.LoginBean;
28 import com.fasterxml.jackson.databind.ObjectMapper;
29
30 /**
31  * Test sur la classe IAuthentificationService.
32  */
33 @RunWith(SpringJUnit4ClassRunner.class)
34 @ContextConfiguration(name = "applicationContext", locations = { "classpath:*-context.xml", "file:WebContent/WEB-INF/MVCDispatcher-servlet.xml" })
35 @WebAppConfiguration
36 public class TestUtilisateurWServiseND {
```

On Ajoute cette annotation

On complète sa liste de fichier Spring

Spring MVC : JUnit – Avec MockMvc

- On ajoute deux attributs à notre test unitaire
 - Le premier de type `WebApplicationContext` qui sera en `@Autowired`. Cet objet va porter le contexte Spring.
 - Le second de type `MockMvc`. Cet objet va nous permettre de tester nos WS
- On ajoute aussi une méthode en `@Before` (ou `@BeforeEach`) qui va initialiser notre objet `MockMvc`

```
42 @Autowired
43 private WebApplicationContext webApplicationContext;
44
45 private MockMvc mockMvc;
46
48 * Fabrication de notre mock MVC
50 @Before
51 public void setup() {
52     this.mockMvc = MockMvcBuilders.webAppContextSetup(this.webApplicationContext).build();
53 }
```

Spring MVC : JUnit – Avec MockMvc

- En Spring Boot uniquement, vous pouvez aussi faire usage de l'annotation **@AutoConfigureMockMvc** qui remplace
 - l'annotation @WebAppConfiguration
 - Et l'attribut WebApplicationContext
 - Et la méthode @Before (ou @BeforeEach) d'initialisation du MockMvc
 - ❑ Il passe en @Autowired

```
@SpringBootTest // Spring Boot
@Transactional
@Rollback(true)
@AutoConfigureMockMvc
class WebTest {

    @Autowired
    private MockMvc mockMvc;

    // Plus de @Before
    ...
}
```

Spring MVC : JUnit – Avec MockMvc

```
@Test
void testAdd01() throws Exception {
    // Connect as Lunch Lady
    var result = super.logMeInAsLunchLady();

    var dto = new ConstraintDtoIn();
    dto.setMaximumOrderPerDay(Integer.valueOf(20));
    dto.setRateVAT(Float.valueOf(20F));
    dto.setOrderTimeLimit("11:00:00");
    var dtoAsJsonString = this.mapper.writeValueAsString(dto);

    // Appel
    // - URL_ADD : Url sans http:// ..., juste l'URL de @RequestMapping
    // - Dans cet exemple on y ajoute le token JWT issu de l'authentification
    result = this.mockMvc.perform(MockMvcRequestBuilders.put(ConstraintRestControllerTest.URL_ADD)
        .contentType(MediaType.APPLICATION_JSON_VALUE).content(dtoAsJsonString)
        .header(SecurityConstants.TOKEN_HEADER, super.getJWT(result)));

    // On veut un status ok
    result.andExpect(MockMvcResultMatchers.status().isOk());
    // On veut que dans le flux Json de retour il y ait un id
    result.andExpect(MockMvcResultMatchers.jsonPath("$.id").exists());
    // mapper est l'ObjectMapper de Jackson que l'on a @Autowired pour nos tests
    var dtoOut = this.mapper.readValue(result.andReturn().getResponse().getContentAsString(),
        ConstraintDtoOut.class);

    var entity = this.constraintService.find(dtoOut.getId());
    Assertions.assertNotNull(entity, "Result must exist");
    Assertions.assertNotNull(entity.getId(), "Result must have an id");
}
```

Spring MVC : JUnit – Avec MockMvc

- La méthode **perform** de l'objet MockMvc permet de simuler l'envoi d'une requête HTTP.
- Elle prend en paramètre la commande HTTP
 - Get, post, put, ...
 - Et l'URL ⇔ ATTENTION : uniquement celui servant au mapping du WS ⇔ Celui indiqué dans l'annotation @RequestMapping
- On peut la compléter par des informations supplémentaires
 - Ici on précise que l'on va envoyer du Json + le header avec le JWT

Spring MVC : JUnit – Avec MockMvc

- La méthode **perform** retourne un objet **ResultActions**
- Sur cet objet vous pouvez appeler d'autres méthodes de validation via l'appel à **andExpect()**
- Si vous voulez récupérer le **status** ou la **réponse**, vous devrez passer par la méthode **andReturn()** qui vous donnera un objet **MvcResult**
 - Il suffit ensuite d'appeler par exemple **getResponse().getContentAsString()** pour récupérer son flux Json de réponse
 - Ou **getResponse().getStatus()** pour récupérer le HttpStatus code
 - Ou **getResponse().getRequest().getSession()** pour récupérer une HttpSession

Spring MVC : JUnit – Avec MockMvc

- Dans le cas où vous avez fait usage de l'objet HttpSession, le code devient un peu plus complexe
- Méthode `andReturn().getRequest().getSession()` sur notre objet `ResultActions`

- 1er appel REST

```
// On appelle le web service en POST, en envoyant du JSON
// Attention : il n'y a plus d'URL complet (http://....)
// Juste l'URL du web service <=> @RequestMapping
ResultActions result = this.mockMvc.perform(MockMvcRequestBuilders.post("/authentifier")
    .contentType(MediaType.APPLICATION_JSON).content(loginBeanAsJson));
```

- Récupération de la session

```
// On récupère la session pour la repasser à la requête suivante
// On fait cela car notre WS se sert de HttpSession
session = result.andReturn().getRequest().getSession();
```

- 2e appel avec la session

```
// On appelle le web service en GET, en remettant la session
ResultActions result = this.mockMvc
    .perform(MockMvcRequestBuilders.get("/comptes").session((MockHttpSession) session));
```

Spring MVC : JUnit – Avec MockMvc

31

```
93+ * Usage des mock.
95- @Test
96 public void testListerCompteOk() {
97     TestUtilisateurWServeurND.LOG.trace("Test - testListerCompteOk");
98     // Bean qui sert à l'authentification
99     LoginBean loginBean = new LoginBean();
100     loginBean.setLogin("df");
101     loginBean.setPassword("df");
102     // Objet session qui permet de propager la session dans nos requetes qui
103     // suivent l'authentification
104     HttpSession session = null;
105     UtilisateurEntity objRetour = null;
106     // Objet -> Json en Jackson
107     ObjectMapper mapper = new ObjectMapper();
108     try {
109         String loginBeanAsJson = mapper.writeValueAsString(loginBean);
110
111         // On appelle le web service en POST, en envoyant du JSON
112         // Attention : il n'y a plus d'URL complet (http://....)
113         // Juste l'URL du web service <=> @RequestMapping
114         ResultActions result = this.mockMvc.perform(MockMvcRequestBuilders.post("/authentifier")
115             .contentType(MediaType.APPLICATION_JSON).content(loginBeanAsJson));
116         // On test que le resultat est bien 200
117         result.andExpect(MockMvcResultMatchers.status().isOk());
118         // Avec un flux JSON qui contient une propriete id qui vaut 1
119         result.andExpect(MockMvcResultMatchers.content().contentType(MediaType.APPLICATION_JSON_VALUE))
120             .andExpect(MockMvcResultMatchers.jsonPath("$.id").value(Integer.valueOf(1)));
121         // On recupere le session pour la repasser a la requete suivante
122         // On fait cela car notre WS se sert de HttpSession
123         session = result.andReturn().getRequest().getSession();
124         // Recuperation du flux en JSON
125         String fullStringResult = result.andReturn().getResponse().getContentAsString();
126         TestUtilisateurWServeurND.LOG.debug("JSON Result is {}", fullStringResult);
127         // Json -> Object en Jackson
128         objRetour = mapper.readValue(fullStringResult, UtilisateurEntity.class);
129     } catch (Exception e) {
130         TestUtilisateurWServeurND.LOG.error("Erreur", e);
131         Assert.fail(e.getMessage());
132     }
```


Spring MVC : JUnit – Avec MockMvc

```
133 Assert.assertNotNull("La session ne doit pas être null", session);
134 Assert.assertNotNull("La session doit contenir le user id", session.getAttribute("userId"));
135 Assert.assertNotNull("L'objet traduit du Json ne doit pas être null", objRetour);
136 Assert.assertEquals("Le user id est le même que celui du flux JSON", objRetour.getId(),
137     session.getAttribute("userId"));
138
139 // On va maintenant récupérer les comptes de l'utilisateur connecté
140 try {
141     // On appelle le web service en GET, en remettant la session
142     ResultActions result = this.mockMvc
143         .perform(MockMvcRequestBuilders.get("/comptes").session((MockHttpSession) session));
144     // On teste que le résultat est bien 200
145     result.andExpect(MockMvcResultMatchers.status().isOk());
146     // Avec un flux JSON qui contient une propriété liste qui n'est pas
147     // null
148     result.andExpect(MockMvcResultMatchers.content().contentType(MediaType.APPLICATION_JSON_VALUE))
149         .andExpect(MockMvcResultMatchers.jsonPath("$.liste").value(CoreMatchers.notNullValue()));
150     // Récupération du flux en JSON
151     String fullStringResult = result.andReturn().getResponse().getContentAsString();
152     TestUtilisateurWServiceND.LOG.debug("JSON Result is {}", fullStringResult);
153 } catch (Exception e) {
154     TestUtilisateurWServiceND.LOG.error("Erreur", e);
155     Assert.fail(e.getMessage());
156 }
```


Spring MVC : JUnit – Avec MockMvcTester

- Le MockMvc a été secondé depuis le Spring 6 (Spring Boot 3.4) par le **MockMvcTester**
 - Il manipule le *MockMvc* et est plus fluide dans son utilisation, sa manipulation
 - ❑ Plus besoin du MockMvcRequestBuilders
 - ❑ Plus besoin du MockMvcResultMatchers
 - Son objectif est le même
- Vous êtes libre d'utiliser l'un ou l'autre
- Il n'est pas conseillé de prendre les deux dans le même projet pour des questions de lisibilité / maintenabilité

Spring MVC : JUnit – Avec MockMvcTester

```
@Test
void testAdd01() throws Exception {
    // Connect as Lunch Lady
    var result = super.logMeInAsLunchLady();

    var dto = new ConstraintDtoIn();
    dto.setMaximumOrderPerDay(Integer.valueOf(20));
    dto.setRateVAT(Float.valueOf(20F));
    dto.setOrderTimeLimit("11:00:00");
    var dtoAsString = this.mapper.writeValueAsString(dto);

    // Appel
    // - URL_ADD : Url sans http:// ..., juste l'URL de @RequestMapping
    // - Dans cet exemple on y ajoute le token JWT issu de l'authentification
    var testResult = this.mockMvcTester.put(
        .uri(ConstraintRestControllerTest.URL_ADD)
        .contentType(MediaType.APPLICATION_JSON_VALUE)
        .content(dtoAsString)
        .header(SecurityConstants.TOKEN_HEADER, super.getJWT(result))
        .exchange();

    // On veut un status ok et un id dans le flux Json de retour
    Assertions.assertThat(testResult)
        .hasStatusOk()
        .bodyJson()
        .hasPath("$.id");

    // Conversion directe du corps en ConstraintDtoOut (plus besoin du mapper manuel)
    var dtoOut = testResult.getResponse()
        .getContentAsString();
    var dtoOutObj = this.mapper.readValue(dtoOut, ConstraintDtoOut.class);

    var entity = this.constraintService.find(dtoOutObj.getId());
    Assertions.assertNotNull(entity, "Result must exist");
    Assertions.assertNotNull(entity.getId(), "Result must have an id");
}
```

```
@Test
void testAdd01() throws Exception {
    // Connect as Lunch Lady
    var result = super.logMeInAsLunchLady();

    var dto = new ConstraintDtoIn();
    dto.setMaximumOrderPerDay(Integer.valueOf(20));
    dto.setRateVAT(Float.valueOf(20F));
    dto.setOrderTimeLimit("11:00:00");
    var dtoAsString = this.mapper.writeValueAsString(dto);

    // Appel
    // - URL_ADD : Url sans http:// ..., juste l'URL de @RequestMapping
    // - Dans cet exemple on y ajoute le token JWT issu de l'authentification
    result = this.mockMvc.perform(
        MockMvcRequestBuilders.put(ConstraintRestControllerTest.URL_ADD)
        .contentType(MediaType.APPLICATION_JSON_VALUE)
        .content(dtoAsString)
        .header(SecurityConstants.TOKEN_HEADER, super.getJWT(result)));

    result.andExpect(MockMvcResultMatchers.status().isOk());
    result.andExpect(MockMvcResultMatchers.jsonPath("$.id").exists());

    var dtoOut = this.mapper.readValue(result.andReturn().getResponse().getContentAsString(),
        ConstraintDtoOut.class);

    var entity = this.constraintService.find(dtoOut.getId());
    Assertions.assertNotNull(entity, "Result must exist");
    Assertions.assertNotNull(entity.getId(), "Result must have an id");
}
```

Spring MVC : JUnit – Avec RestTemplate

- Pour tester vos contrôleurs WS il vous faut simuler/créer des requêtes HTTP
- En Spring, l'objet `org.springframework.web.client.RestTemplate` vous permettra de le faire très simplement
- Il suffit d'instancier un `RestTemplate` au moment où vous souhaitez envoyer votre requête
- En fonction des méthodes du `RestTemplate` que vous utiliserez cela enverra
 - un GET, PUT, POST, ...
 - avec des paramètres
 - avec du JSON / XML
 - ...

Spring MVC : JUnit – Avec RestTemplate

- Point important : pour fonctionner, l'objet RestTemplate a besoin que votre web service **soit déployé** sur un serveur

➤ Méthodes du RestTemplate

- Premier paramètre : URL
- Second paramètre de type Class : le type de retour
- Second paramètre de type Objet : l'élément à envoyer
- Dernier paramètre ... : Pour boucher les trous dans l'URL

HTTP	RESTTEMPLATE
DELETE	<code>delete(String, String...)</code>
GET	<code>getForObject(String, Class, String...)</code>
HEAD	<code>headForHeaders(String, String...)</code>
OPTIONS	<code>optionsForAllow(String, String...)</code>
POST	<code>postForLocation(String, Object, String...)</code>
PUT	<code>put(String, Object, String...)</code>

Spring MVC : JUnit – Avec RestTemplate

- Exemple d'utilisation (GET) : envoyer des paramètres => récupérer un flux JSON sous forme de String

```
27+ * Test
29- @Test
30 public void testAuthentifierOk() {
31     TestUtilisateurWService.LOG.trace("Test - testAuthentifierOk");
32     final String login = "df";
33     final String pwd = "df";
34     RestTemplate restTemplate = new RestTemplate();
35     String resu = null;
36     try {
37         resu = restTemplate.getForObject(
38             "http://localhost:8080/netbank/rest/authentifier/byparam?login={login}&password={pwd}",
39             String.class, login, pwd);
40     } catch (RestClientException e) {
41         TestUtilisateurWService.LOG.error("Erreur", e);
42         Assert.fail(e.getMessage());
43     }
44     TestUtilisateurWService.LOG.debug("Test - testAuthentifierOk - {}", resu);
45 }
```

Spring MVC : JUnit – Avec RestTemplate

- Exemple d'utilisation (POST) : envoyer un flux JSON => Récupérer un flux
- Dans cet exemple, le framework Jackson est une dépendance du projet

```
42 @Test
43 public void testAuthentifierOk() {
44     TestUtilisateurWService.LOG.trace("Test - testAuthentifierOk");
45     RestTemplate restTemplate = new RestTemplate();
46     LoginBean loginBean = new LoginBean();
47     loginBean.setLogin("df");
48     loginBean.setPassword("df");
49     try {
50         // Envoie de login/pwd en JSON en post
51         restTemplate.postForLocation(TestUtilisateurWService.AUTHENTIFIER_URL, loginBean);
52     } catch (RestClientException e) {
53         TestUtilisateurWService.LOG.error("Erreur", e);
54         Assert.fail(e.getMessage());
55     }
56 }
```

Spring MVC : JUnit – Avec RestTemplate

- Vous pouvez récupérer la réponse dans son intégralité en passant par la méthode *exchange*
- Dans l'exemple qui suit :
 - On envoie une première requête HTTP en POST avec un objet JSON
 - A travers la réponse, on récupère un JSESSIONID
 - ❑ On fait cela car côté serveur dans cet exemple, le WS fait usage de l'objet HttpSession et on en a besoin pour faire les autres appels (problématique de l'authentification)
 - Dans la seconde requête on demande en GET la liste des comptes de l'utilisateur connecté
 - Dans les deux cas, on obtient un flux JSON

Spring MVC : JUnit – Avec RestTemplate

40

```
91 @Test
92 public void testListerCompteOk2() {
93     RestTemplate restTemplate = new RestTemplate();
94     // Notre bean/model pour le JSON en envoi
95     LoginBean loginBean = new LoginBean();
96     loginBean.setLogin("df");
97     loginBean.setPassword("df");
98     // Indiquons que l'on accepte le json
99     HttpHeaders headers = new HttpHeaders();
100     headers.setAccept(Arrays.asList(MediaType.APPLICATION_JSON));
101     HttpEntity<?> entity = new HttpEntity<>(loginBean, headers);
102
103     ResponseEntity<String> result = restTemplate.exchange(TestUtilisateurWSERVICE.AUTHENTIFIER_URL, HttpMethod.POST,
104         entity, String.class);
105     TestUtilisateurWSERVICE.LOG.debug("Resultat-status {}", result.getStatusCode());
106     TestUtilisateurWSERVICE.LOG.debug("Resultat-flux de retour {}", result.getBody());
107
108     // Afin de lister le contenu du header de la réponse
109     // for (Map.Entry<String, List<String>> elm :
110     // result.getHeaders().entrySet()) {
111     // TestUtilisateurWSERVICE.LOG.debug("{}={}", elm.getKey(),
112     // elm.getValue());
113     // }
114
115     // Pour la seconde requete on recupere le JSESSIONID et on le repousse
116     // On fait cela car notre WS se sert de HttpSession
117     headers.clear();
118     headers.add("Cookie", result.getHeaders().get("Set-Cookie").get(0));
119     headers.setAccept(Arrays.asList(MediaType.APPLICATION_JSON));
120     entity = new HttpEntity<>(null, headers);
121     result = restTemplate.exchange(TestUtilisateurWSERVICE.LISTER_CPT_URL, HttpMethod.GET, entity, String.class);
122     TestUtilisateurWSERVICE.LOG.debug("Resultat-status {}", result.getStatusCode());
123     TestUtilisateurWSERVICE.LOG.debug("Resultat-flux de retour {}", result.getBody());
124 }
```


Spring MVC : JUnit – Avec RestClient

- **RestClient** est l'équivalent de RestTemplate.
- Sa vocation est exactement la même, il devrait, un jour le *remplacer*
- On trouve souvent la différence '*virtuelle*' suivante
 - RestClient : call API externe
 - RestTemplate : call API interne

Spring MVC : JUnit – Avec RestClient

➤ Exemple :

```
RestClient restClient = RestClient.builder()  
                                .baseUrl("https://api.example.com")  
                                .build();  
  
User user = restClient.get()  
                    .uri("/users/{id}", 1)  
                    .retrieve()  
                    .body(User.class);
```

Spring MVC : JUnit – Avec WebClient

- **WebClient** est une autre classe du Spring qui a pour objectif de réaliser des appels HTTP
- C'est une classe **plus moderne** que RestTemplate
- La différence majeure réside dans le fait que **WebClient** est **full asynchrone** alors que RestTemplate est synchrone
 - Pour l'utiliser vous aurez besoin d'ajouter la dépendance vers spring-boot-starter-webflux

Spring MVC : JUnit – Avec WebClient

➤ Exemple

```
var client = WebClient.create(WebReactiveClient.ROOT_URL);  
Mono<UtilisateurEntity> resu = client.get().uri(uriBuilder -> uriBuilder  
    .path(WebReactiveClient.AUTH_URL + "byurl/{email}/{pwd}")  
    .build(MongoKeyHandler.USER_EMAIL, MongoKeyHandler.USER_PWD))  
    .retrieve()  
    .bodyToMono(UtilisateurEntity.class)  
    .timeout(timeoutDuration);
```

Spring MVC : JUnit – RestTemplate / WebClient / RestClient

45

- Ni RestTemplate, ni WebClient, ni RestClient ne sont pensés pour être utilisés en JUnit (pour faire des tests)
- Leur rôle est d'être un client HTTP **normal**
- Par contre, pour les TU, dans un usage de TU vous trouverez :
 - WebTestClient
 - TestRestTemplate
 - @RestTestClient

Spring MVC : JUnit – WebTestClient

➤ Exemple

```
@SpringBootTest
class HelloControllerTest {

    private final WebTestClient client =
        WebTestClient.bindToController(new HelloController()).build();

    @Test
    void shouldReturnHelloMessage() {
        client.get()
            .uri("/api/hello")
            .exchange()
            .expectStatus().isOk()
            .expectHeader().contentType("application/json")
            .expectBody()
            .jsonPath("$.message").isEqualTo("Hello from Spring");
    }
}
```

```
@RestController
@RequestMapping("/api")
public class HelloController {

    @GetMapping("/hello")
    public Map<String, String> hello() {
        return Map.of("message", "Hello from Spring");
    }
}
```

Spring MVC : JUnit – TestRestTemplate

➤ Exemple

```
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
class HelloControllerTest {

    @Autowired
    private TestRestTemplate restTemplate;

    @LocalServerPort
    private int port;

    @Test
    void shouldReturnHelloMessage() {
        ResponseEntity<Map> response =
            restTemplate.getForEntity("http://localhost:" + port + "/api/hello", Map.class);

        assertEquals(HttpStatus.OK, response.getStatusCode());
        assertNotNull(response.getBody());
        assertEquals("Hello from Spring", response.getBody().get("message"));
    }
}
```

Spring MVC : JUnit – @RestClientTest

- Exemple : Ici notre service fait usage de RestClient pour faire un call vers une API externe

```
@Service
public class JokeClient {

    private final RestClient restClient;

    public JokeClient(RestClient.Builder builder) {
        this.restClient = builder
            .baseUrl("https://api.example.com")
            .build();
    }

    public JokeResponse getRandomJoke() {
        return restClient.get()
            .uri("/jokes/random")
            .retrieve()
            .body(JokeResponse.class);
    }
}
```


Spring MVC : JUnit – @RestClientTest

- Notre TU veut tester notre service SANS faire un vrai call à l'API externe.

```
@RestClientTest(JokeClient.class)
class JokeClientTest {

    @Autowired
    private JokeClient jokeClient;

    // Objet que nous allons éduquer pour notre @RestClientTest
    @Autowired
    private MockRestServiceServer server;

    @Test
    void shouldReturnMappedJokeResponse() {
        // Flux de retour décidé par nos soins
        String json = ""
            {
                "id": "abc123",
                "value": "Chuck Norris can unit test entire applications with one assert."
            }
            """";

        // Education du serveur 'auto'injecte dans le RestClient
        server.expect(requestTo("https://api.example.com/jokes/random"))
            .andExpect(method(HttpMethod.GET))
            .andRespond(withSuccess(json, APPLICATION_JSON));

        // Notre test maintenant que l'éducation est terminée
        JokeResponse response = jokeClient.getRandomJoke();

        assertEquals("abc123", response.id());
        assertEquals("Chuck Norris can unit test entire applications with one assert.", response.value());
    }
}
```

Spring MVC : JUnit – MockRestServiceServer

- MockRestServiceServer : permet de tester nos API Rest sans déployer l'application
- Il devra donc s'éduquer avant son utilisation
- Il peut se brancher sur un
 - RestClient : `@RestClientTest`
 - RestTemplate : `MockRestServiceServer.createServer(restTemplate);`
- Ne peut PAS se brancher sur
 - WebClient, WebTestClient, TestRestTemplate

Spring MVC : JUnit – Avec MockBean

- Dans le cas où vous souhaitez tester un service métier qui dépend d'un repository, il est de bon ton de mocker le repository afin de garantir que seul le service est testé.
 - Un autre avantage de mocker le repository est que vous n'avez pas besoin d'une base de données
 - ❑ A noter qu'en Spring il est très simple de monter des repository sur une base mémoire H2



Spring MVC : JUnit – Avec MockBean

- Pour ce faire vous pouvez faire usage de l'annotation **@MockBean** du Spring.
 - Replacée par **@MockitoBean** à partir de Spring 6.2+ (Spring Boot 3.4.x)
- Elle se place dans votre classe de test sur l'attribut que vous souhaitez mocker (et qui est nécessaire à l'objet que vous voulez vraiment tester)
- Cette annotation est l'équivalent de **@InjectMocks** de Mockito
 - Que vous ne devriez **jamais** utiliser en Spring pour éviter tout effet de bord

Spring MVC : JUnit – Avec MockBean

➤ Exemple (ici en Spring Boot 2.2 et JUnit 5)

```
@SpringBootTest  
public class StatServiceTest {
```

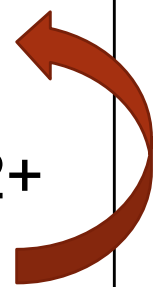
```
    @Autowired  
    private IStatService statService;
```

```
    @MockBean // ou @MockitoBean en v6.2+  
    private IStatRepository statRepository;
```

Objet qui sera vraiment testé



Sera mockée puis
automatiquement
injectée dans



Spring MVC : JUnit – @DataJpaTest

- L'annotation de classe **@DataJpaTest** vous permet de tester les couches @Repository **UNIQUEMENT** en **Spring Data** (quand vous héritez par exemple de JpaRepository)
- Elle va se charger de 'mock' la couche base de données
 - Donc, si vous aviez une DataSource en MySQL elle va en faire une autre
- C'est une manière de faire du H2 sans faire du H2 explicitement

Spring MVC : JUnit – @DataJpaTest

- Tous vos @Test seront automatiquement dans un @Transactional qui sera rollbacké
- SEUL les éléments du repository visé seront chargés en mémoire pour faire le test
 - En principe, c'est plus rapide qu'un test classique
- Cette annotation **REPLACE** @SpringBootTest
- Notez qu'il est souvent nécessaire de faire usage d'un @ActiveProfiles avec @DataJpaTest
 - Par exemple, pour y faire un spring.jpa.hibernate.ddl-auto=create

Spring MVC : JUnit – @DataJpaTest

➤ Exemple

```
@DataJpaTest
class UserRepositoryTest {

    @Autowired
    private UserRepository userRepository;

    @Test
    void shouldFindUserByEmail() {
        User user = new User();
        user.setEmail("test@example.com");
        userRepository.save(user);

        User found = userRepository.findByEmail("test@example.com");

        assertNotNull(found);
    }
}
```


Spring Boot 3.1 et les conteneurs



- Depuis Spring Boot 3.1 (juin 2023) vous pouvez piloter/démarrer/paramétrer un (ou plusieurs) conteneur docker comme si vous aviez codé un docker compose.yaml
- Vous devez avoir un docker installé sur votre machine pour que cela fonctionne
- Le point vraiment intéressant étant que c'est **votre code** Java qui **va piloter** la configuration du conteneur

Spring Boot 3.1 et les conteneurs



- Pour le moment, seuls les conteneurs suivants sont utilisables :
- Cassandra
 - Couchbase
 - Elasticsearch
 - En JDBC ou R2DBC : Oracle, Mariadb, SQL Server, MySql, Postgres
 - Mongo
 - Flyway
 - Kafka
 - Liquibase
 - Neo4J
 - Rabbit
 - Redis
 - Zipkin

Spring Boot 3.1 et les conteneurs



➤ Indiquez les deux dépendances Maven suivantes :

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-testcontainers</artifactId>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.testcontainers</groupId>
  <artifactId>junit-jupiter</artifactId>
  <artifactId>testcontainers-junit-jupiter</artifactId>
  <scope>test</scope>
</dependency>
```

← Spring Boot < 4.1

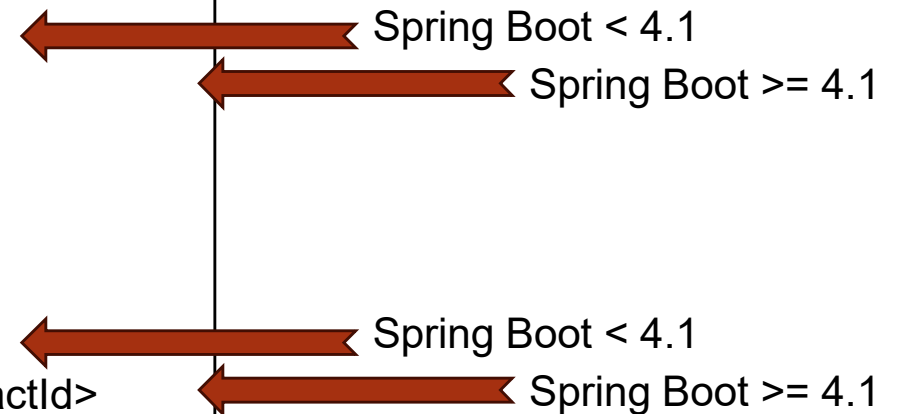
← Spring Boot >= 4.1

Spring Boot 3.1 et les conteneurs



- Puis, en fonction des conteneurs que vous désirez manipuler, indiquez la dépendance Maven vers ce/ces derniers :

```
<dependency>
  <groupId>org.testcontainers</groupId>
  <artifactId>mysql</artifactId>
  <artifactId>testcontainers-mysql</artifactId>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.testcontainers</groupId>
  <artifactId>postgresql</artifactId>
  <artifactId>testcontainers- postgresql </artifactId>
  <scope>test</scope>
</dependency>
```



<https://mvnrepository.com/search?q=org.testcontainers>

Spring Boot 3.1 et les conteneurs



➤ Côté code, dans les Tests Unitaires

➡ Placer sur la classe de test l'annotation **@Testcontainers**

```
@SpringBootTest
@Testcontainers
class IConstraintDaoTest {
```

➡ Déclarez un attribut statique de type *Container*

```
@Container
static MySQLContainer<?> mysql = new MySQLContainer<>("mysql:8.0.32"); // SB < 4.1
static MySQLContainer mysql = new MySQLContainer("mysql:8.0.32"); // SB >= 4.1
```

❑ Notez que nous passons l'image tag dans le paramètre du constructeur du *Container*

❑ ATTENTION à vos imports sur SB 4.1+

Spring Boot 3.1 et les conteneurs



➤ Vous pouvez customiser votre conteneur :

```
static MySQLContainer mysql = new MySQLContainer("mysql:8.0.32")
    .withDatabaseName("dbName")
    .withUsername("user")
    .withPassword("userPwd")
    .withCopyFileToContainer(MountableFile.forClasspathResource("insert.sql"),
        "/docker-entrypoint-initdb.d/insert.sql")
    .withEnv("MYSQL_ALLOW_EMPTY_PASSWORD", "no")
    .withEnv("MYSQL_ROOT_PASSWORD", "root");
```

Spring Boot 3.1 et les conteneurs



- Concernant la configuration, du conteneur vers Spring Boot, vous pouvez faire usage de l'annotation **@ServiceConnection** qui se place avec **@Container**

```
@Container
```

```
@ServiceConnection
```

```
static MySQLContainer mysql = new MySQLContainer("mysql:8.0.32").withXxx()...
```

- Cette annotation injectera la configuration du conteneur dans votre Spring Boot
 - ➡ Donc, pas besoin de réaliser une configuration spéciale côté Spring Boot

Spring Boot 3.1 et les conteneurs



- Après le lancement du test, soyez patient, les conteneurs démarrent
 - 'les' car Spring Boot va en lancer plusieurs, dont celui que vous avez demandé
 - Tant qu'ils vivent, ils seront visibles dans Docker Desktop
- Remarques :
 - Le temps d'exécution des tests ne prend pas en compte le temps d'initialisation de vos conteneurs
 - N'oubliez pas de charger vos configurations (par exemple JPA) pour la bonne cible (dans mon exemple un JPA pour MySQL)
 - Il faut que docker soit installé sur la machine qui va exécuter les tests unitaires

Spring Boot 3.1 et les conteneurs



- Vous pouvez aussi faire une réutilisation des conteneurs lancés/paramétrés par Spring
 - Placez ce dernier dans une interface
 - Faites que vos classes de tests
 - ❑ Implémentent cette interface
 - ❑ Ou, sur votre classe utilisez `@ImportTestcontainers(MonInterface.class)`
- Il existe aussi une autre approche
 - Votre conteneur n'est pas statique
 - Il est déclaré comme un **@Bean**
 - Vous l'injectez **là où** vous souhaitez l'utiliser